1. INTRODUCTION

1.1 INTRODUCTION

In today's competitive business world, customer retention is crucial for long-term success. Businesses often face the challenge of customer churn, where clients end their association with the company. To tackle this, advanced machine learning methods are increasingly used for predictive analytics. The project focuses on building an effective predictive model for client churn using the Gradient Boosting Classifier algorithm. By leveraging this technique, we aim to create a robust model that accurately identifies clients at risk of leaving. The model uses historical customer data, including usage patterns and demographics, to learn patterns associated with churn behavior. Through iterative training and validation, it refines its predictive capabilities, evaluated based on metrics like accuracy and precision. These predictions empower businesses to proactively implement retention strategies, thereby reducing churn rates and enhancing profitability. In essence, employing Gradient Boosting for churn prediction is a strategic move that helps businesses strengthen customer relationships and drive sustainable growth through informed decision-making.

1.2 PROBLEM STATEMENT

Clients leave the bank due to dissatisfaction of the transaction fees, relocation of customer, age-based preferences and reduced account balance which increases the marketing cost to acquire new clients and impacting profitability and growth of the bank.

1.3 PROJECT OBJECTIVE

The aim for client churn prediction using Gradient Boosting Classifier is to develop accurate and effective models that can predict which bank's customers are likely to leave in the future.

1.4 SCOPE OF THE PROECT

1.4.1 EXISTING SYSTEM

The study introduces a new Ensemble Deep Learning model called AOA-SBLSTM-RNN, specifically designed to understand information about insurance customers.. It uses AOA to extract important features from the data. By processing three different insurance datasets, it cleans up the raw data, identifies customers likely to switch using SBLSTM-RNN, and fine-tunes the model using IGSA.The results indicate that this approach outperforms others in predicting customer churn in the insurance industry.

1.4.2 PROPOSED SYSTEM

The client churn prediction system utilizes the powerful Gradient Boosting Classifier algorithm to enhance accuracy in forecasting customer attrition. This machine learning approach employs boosting and sequential integration to create a robust predictive model. The standard steps include handling missing values, text-to-number conversion, and normalization. Key client features, including demographics, usage patterns, and transaction history, undergo comprehensive analysis for accurate predictions. Through iterative optimization, the Gradient Boosting Classifier effectively identifies potential churners, enabling proactive customer retention strategies. The success of Gradient

Boosting lies in its ability to handle complex relationships within the data, making it a popular choice for accurate and efficient churn prediction in various industries.

1.5 SOFTWARE LIFE CYCLE MODELS

Software Life Cycle

The period of time that start when a software is conceived and end when the product no longer available for use. The Software Life Cycle typically includes:

- Requirement phase
- Design phase
- Implementation phase
- Test phase
- Installation and check out phase
- Operation and maintenance phase

Requirement Phase

The goal is to understand exact requirements of the customer and document them properly.

Design Phase

The goal is to transform the requirements specification into a structure that suitable for implementation in some programming language.

Implementation Phase

Design representations are translated into actual programs (Design is implemented).

Test Phase

After a system has been developed, it is very important to check if it is the customer requirements. For this purpose, testing of the system is done. The major activities are centered around the examination and modification of the code.

Installation and Check out Phase

Implementation of system means putting up system on user's site. Acceptance testing ensures that the system meets all the requirements. If it fulfills the needs then the system is accepted by the customer and put into use.

Operation and Maintenance Phase

The purpose of this phase is to preserve the value of software over time. This phase may span for 5 to 50 years whereas development may be 1 to 3 years. Software and Hardware maintenance is a very broad activity that includes error correction, enhancement of capabilities, deletion of obsolete, capabilities, and optimization. This is very crucial for the system's life.

1.5.1 TYPES

The common life cycle models are

- Spiral model
- Waterfall modeL
- Throwaway prototyping model
- Evolutionary prototyping model
- Incremental/iterative development
- WIN/ WIN Spiral model

1.5.2 ITERATIVE MODEL

The iterative model in software development is an iterative and flexible approach that breaks down the project into smaller, manageable segments or increments. Each increment represents a portion of the software's functionality and can be developed, tested, and delivered independently. Unlike traditional waterfall models, where the entire project is completed in a single phase, the iterative model allows for the continuous addition of features or improvements over time.

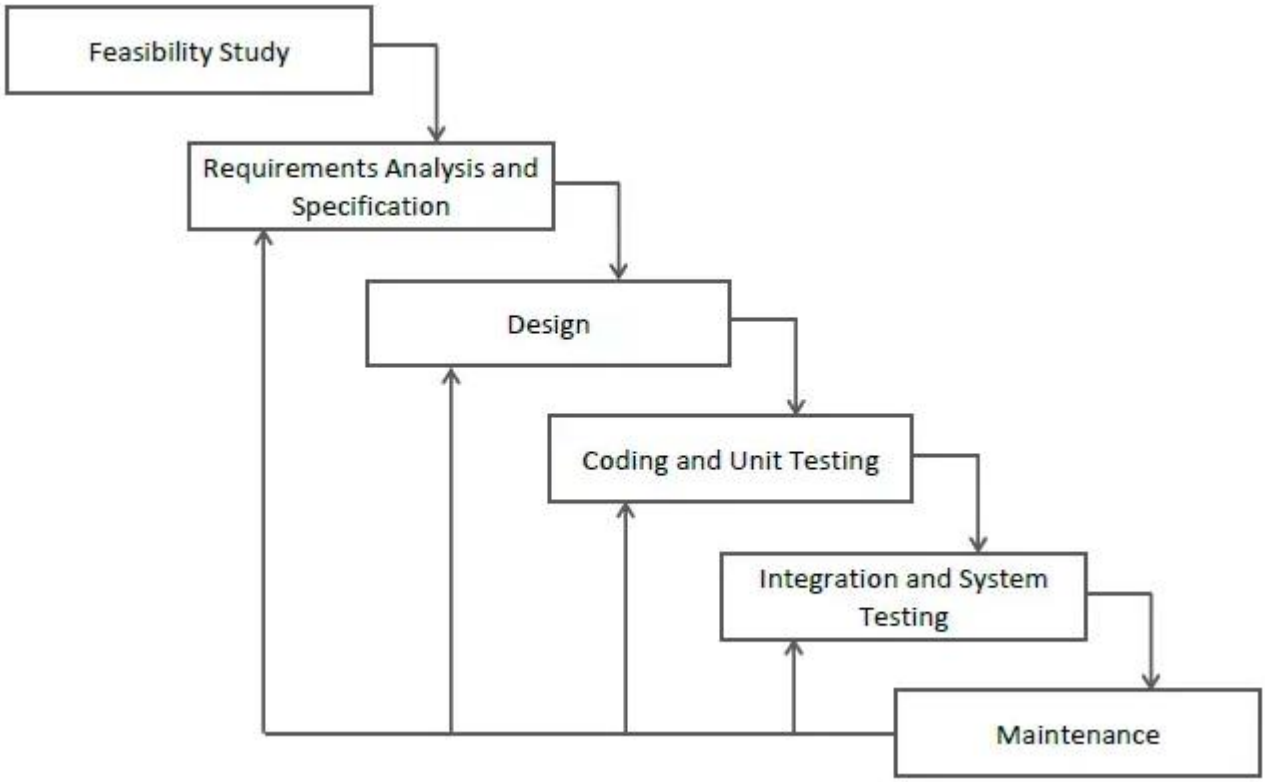

Fig.1.5.1 Iterative model

1.5.3 REASON FOR CHOOSING THIS MODEL

- Product definition is stable.
- Technology is understood.
- There are no ambiguous requirements

2. LITERATURE REVIEW

2.1 INTRODUCTION

The literature review has been done to know about the different types of communications used according to time and technology changes.

2.2 LITERATURE REVIEW

[1]. N. Jajam and N. P. Challa, "Customer churn detection for insurance data using blended logistic regression decision tree algorithm (BLRDT)," Int. J. Intell. Syst. Appl. Eng., vol. 11, no. 1, pp. 72–83, 2023.

The insurance sector is a prominent adopter of data forecasting technologies, leveraging vast amounts of data and catering to an ever-expanding customer base. With clients having the ability to switch insurance providers easily, there's a challenge in accurately predicting churn probability. If predictions fall short, it indicates that the insurance company's strategies may not align with the evolving needs of customers. To address the challenges in churn prediction, the discussion revolves around the application of machine learning (ML) and deep learning (DL) methods. These advanced techniques aim to provide more accurate and reliable churn forecasts.

[2]. Z. Wu, L. Jing, B. Wu, and L. Jin, "A PCA-AdaBoost model for e-commerce customer churn prediction," Ann. Oper. Res., vol. 2, pp. 1–18,2022. [Online].

The research paper titled "A PCA-AdaBoost model for e-commerce customer churn prediction" authored by Z. Wu, L. Jing, B. Wu, and L. Jin was published in the year 2022. The paper likely introduces a predictive modeling approach for predicting customer churn in the e-commerce domain. The aim is to improve the accuracy of predicting customer churn in e-commerce scenarios. The model mentioned is a combination of Principal Component Analysis (PCA) and AdaBoost, indicating the use of dimensionality reduction and ensemble learning techniques.

[3]. A. B. W. Ali, ‘‘Prediction of employee turn over using random forest classifier with intensive optimized PCA algorithm,’’ Wireless Pers. Commun. ,vol. 119, no. 4, pp. 3365–3382, Aug. 2021.

The research paper titled "Prediction of employee turnover using random forest classifier with intensive optimized PCA algorithm" authored by A. B. W. Ali and the research paper was published in the year 2021. The authors use a combination of two techniques: a "Random Forest Classifier" and an advanced version of the "Principal Component Analysis (PCA)" algorithm that has been intensively optimized for better performance. The term "intensive optimized" indicates that the researchers have made significant efforts to improve and fine-tune the PCA algorithm for the specific task of predicting employee turnover.

[4]. A. A. Q. Ahmed and D. Maheswari , “An enhanced ensemble classifier for telecom churn prediction using cost based uplift modelling ,” Int. J. Inf. Technol., vol. 11, no. 2, pp. 381–391, Jun. 2019, doi: 10.1007/s41870-018-0248-3.

In this paper, author Ahmed and Maheswari propose an advanced ensemble classifier for telecom churn prediction, incorporating cost-based uplift modeling. Published in the International Journal of Information Technology, the work addresses telecom industry challenges. The enhanced classifier aims to improve accuracy and efficiency in predicting customer churn. The study contributes to uplift modeling, emphasizing cost considerations. Published in June 2019, the research showcases a novel approach, providing valuable insights for telecom operators seeking effective strategies for customer retention.

2.3 SUMMARY

The papers referred above are being used to learn the different technologies used at different time according to the variations in the specifications.

3. SYSTEM ANALYSIS

Introduction

This chapter contains the detailed description of the requirement used in the entire project. They are segregated into functional and non-functional requirements.

3.1 REQUIREMENT ANALYSIS

3.1.1 FUNCTIONAL REQUIREMENTS

Input

The input of this system is from the user which has given during dataset extraction.

Process

According to the given input, Analyze historical data, extract relevant features, train predictive models (e.g., logistic regression, random forest), evaluate model performance, and deploy the best-performing model to predict future client churn accurately.

Output

Predicting client churn involves analyzing customer behavior to anticipate who is likely to stop using a service or product. This helps businesses take proactive measures to retain customers and maintain their loyalty.

3.1.2 NON FUNCTIONAL REQUIREMENTS

User friendly

Since the project is developed in Python, an open source software makes it easy to write code and upload it to the board.

Platform independent

The Project is implemented using PYTHON, it is Platform independent.

Easy of use

The client churn prediction model is accessible, allowing for seamless usage and enhancing efficiency with increasing user engagement.

Some other non-functional requirements :

- Scalability
- Accuracy
- Interpretability
- Real-time processing
- Seamless integration
- Robustness against outliers

3.1.3 HARDWARE REQUIREMENTS

- Windows 10
- Processor –i3
- RAM-4GB or above
- ROM-512SSD

3.1.4 SOFTWARE REQUIREMENTS

- A database system for storing user interactions and data, such as CSV File
- Python version (3.12 or any other version)
- Sublime Text Code Editor

3.2 MODULE DESCRIPTION

- Project planning and deployment
- Data collection and preprocessing
- Data analysis

- Implementing the Gradient Boosting Classifier Algorithm

3.2.1 PROJECT PLANNING AND DEPLOYMENT

Select Python as the programming language for developing the client churn predictive model. Analyze and choose appropriate machine learning models for churn prediction. Common choices include logistic regression, decision trees, random forests, gradient boosting machines, and neural networks. Use 'pip' to install libraries and frameworks. For example, to install popular libraries like NumPy, Pandas, Scikit-learn and Matplotlib: "pip install numpy pandas". To install and use an integrated development environment (IDE) for a more convenient development experience. Popular Python IDEs include PyCharm, Visual Studio Code, Sublime Text and Jupyter Notebook.

3.2.2 DATA COLLECTION AND PRE-PROCESSING

Data collection for client churn prediction involves gathering relevant customer data such as demographics, purchase history, customer ID, gender, age, balance, and interaction logs. Data preprocessing for missing values involves strategies like imputation, where missing data is estimated or filled in with statistical measures such as mean, median, mode, and standard deviation. Data preprocessing for outlier detection in client churn prediction involves using statistical method like interquartile range (IQR) detection to identify and handle abnormal data points. Scaling ensures numerical features are on similar scales using z-score normalization method, helping machine learning model performance.

3.2.3 DATA ANALYSIS

Exploratory Data Analysis, is the process of analyzing data to understand its patterns, relationships, and distributions to gain insights and inform decision-making. Univariate analysis examines variables individually, using histograms for numerical data and bar plots for categorical data. Bivariate analysis explores relationships between variables,

comparing them with the churn label through scatter plots or correlation matrices. Predicting churn by providing insights into customer behaviour.

Summary

Feature engineering creates new metrics like average transaction amount, aiding in the various requirements used for the project are explained in detail along with their basic purpose. These requirements may be used in the future for other purposes.

4. SYSTEM DESIGN

4.1 DATA FLOW DIAGRAM

DFD is the abbreviation for Data Flow Diagram. The flow of data of a system or a process is represented by DFD. It also gives insight into the inputs and outputs of each entity and the process itself. DFD does not have control flow and no loops or decision rules are present. Specific operations depending on the type of data can be explained by a flowchart. It is a graphical tool, useful for communicating with users, managers and other personnel.

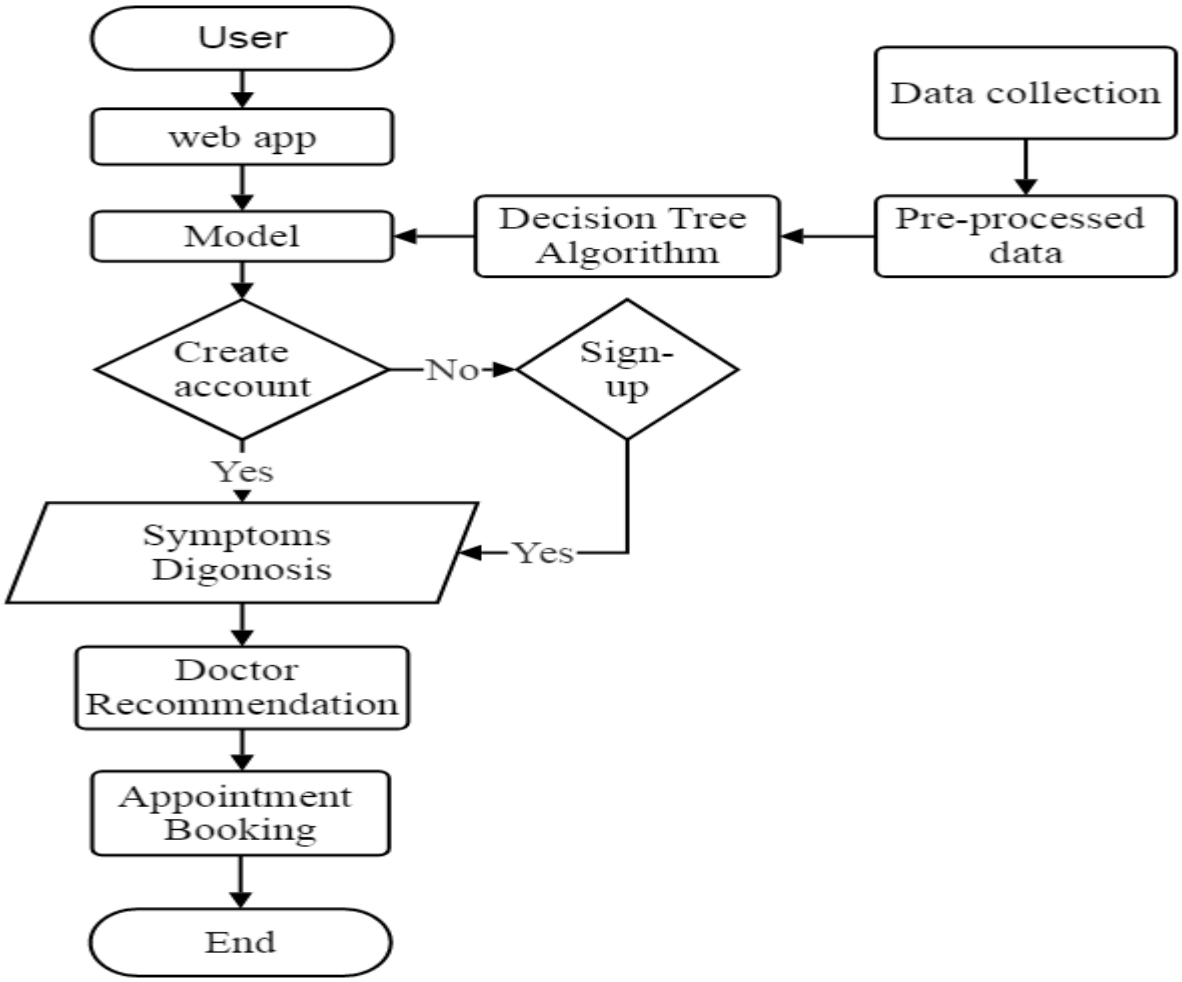

Fig:4.1 Data Flow Diagram for Medimate Messenger

4.2 UML DIAGRAMS

Unified Modeling Language

Unified Modeling Language (UML) is a standardized graphical language. It offers a common language for communication and understanding among stakeholders, including developers, designers, project managers, and clients. UML includes different diagram types, such as class diagrams, use case diagrams, sequence diagrams, and more, each serving a specific purpose in the software development process.

4.2.1 USE CASE DIAGRAM

A use case diagram is a type of Unified Modeling Language (UML) diagram that provides a visual representation of how a software system interacts with its external actors or entities. It illustrates the various use cases, which are specific functionalities or actions that the system can perform, and the relationships between these use cases and the actors. Use case diagrams help stakeholders understand the system's behavior, requirements.

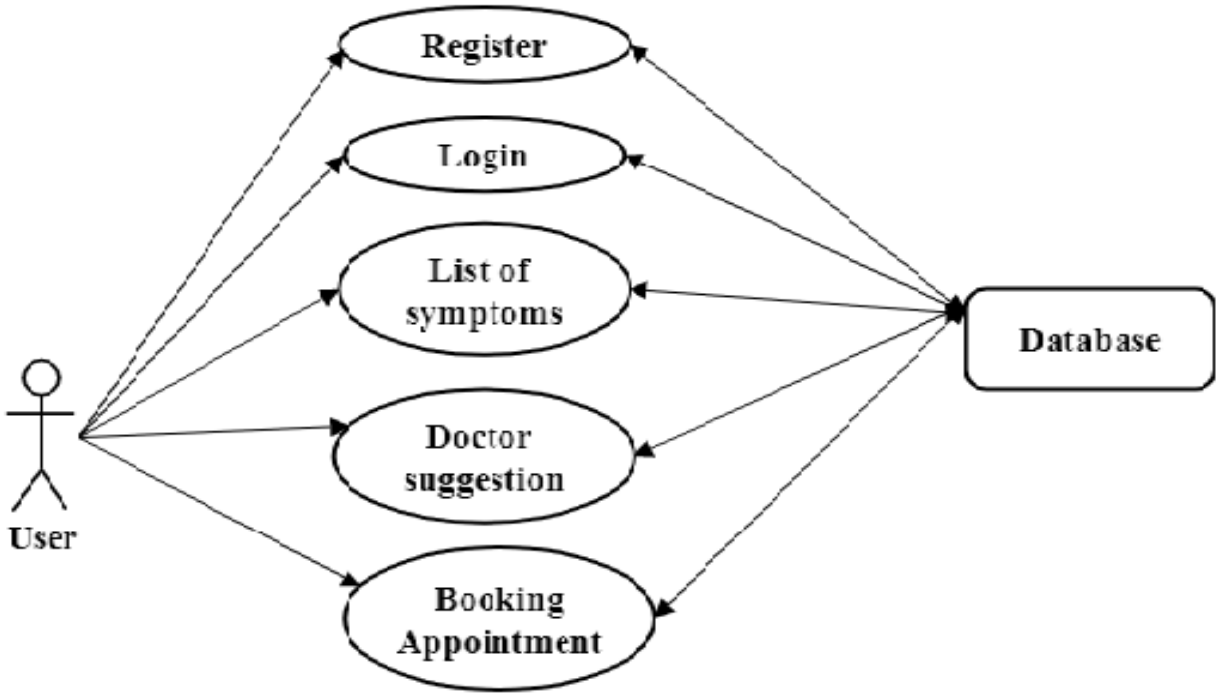

Fig 4.2.1 Use Case Diagram for Medimate Messenger

4.2.2 CLASS DIAGRAM

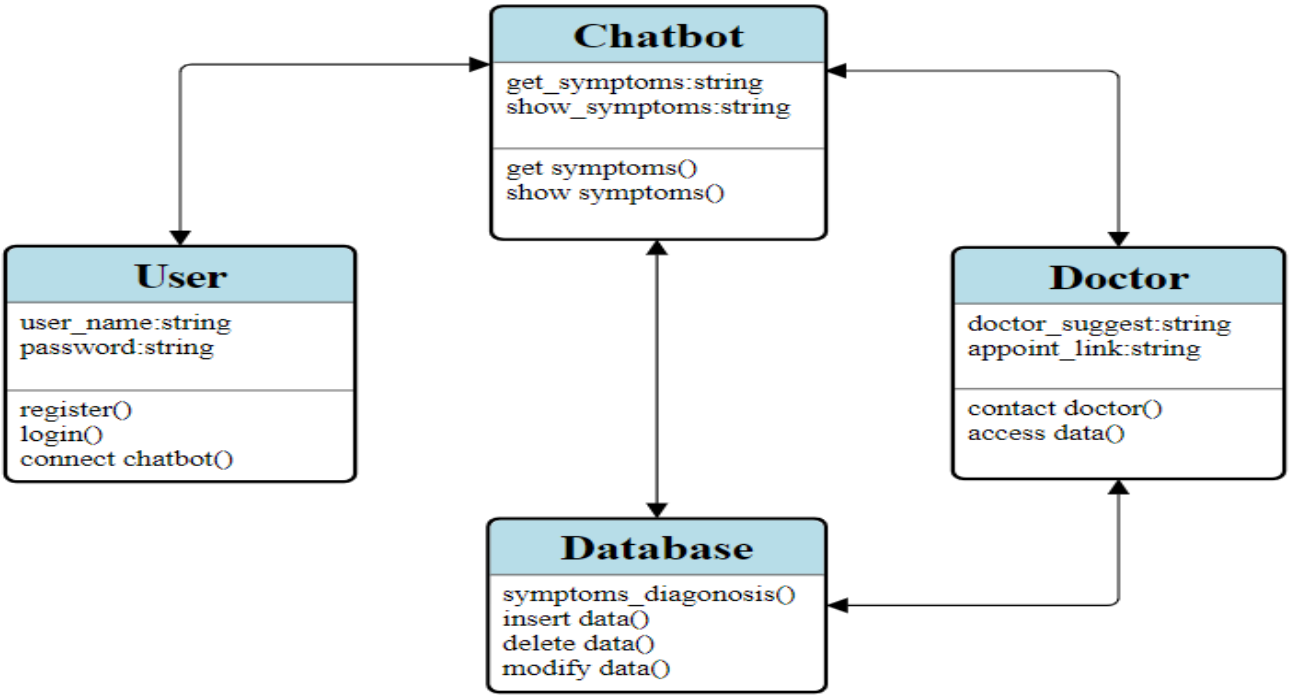

Fig 4.2.2 Class Diagram for Medimate Messenger

4.2.3 SEQUENCE DIAGRAM

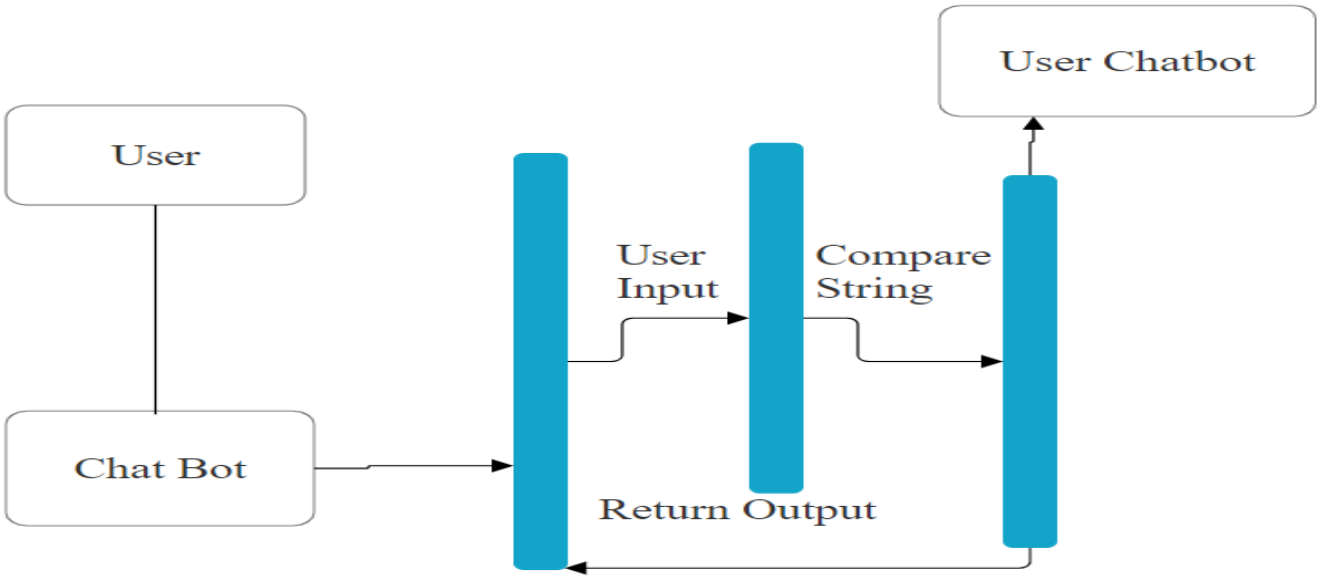

Fig 4.2.3 Sequence Diagram for Medimate Messenge

4.2.4 ACTIVITY DIAGRAM

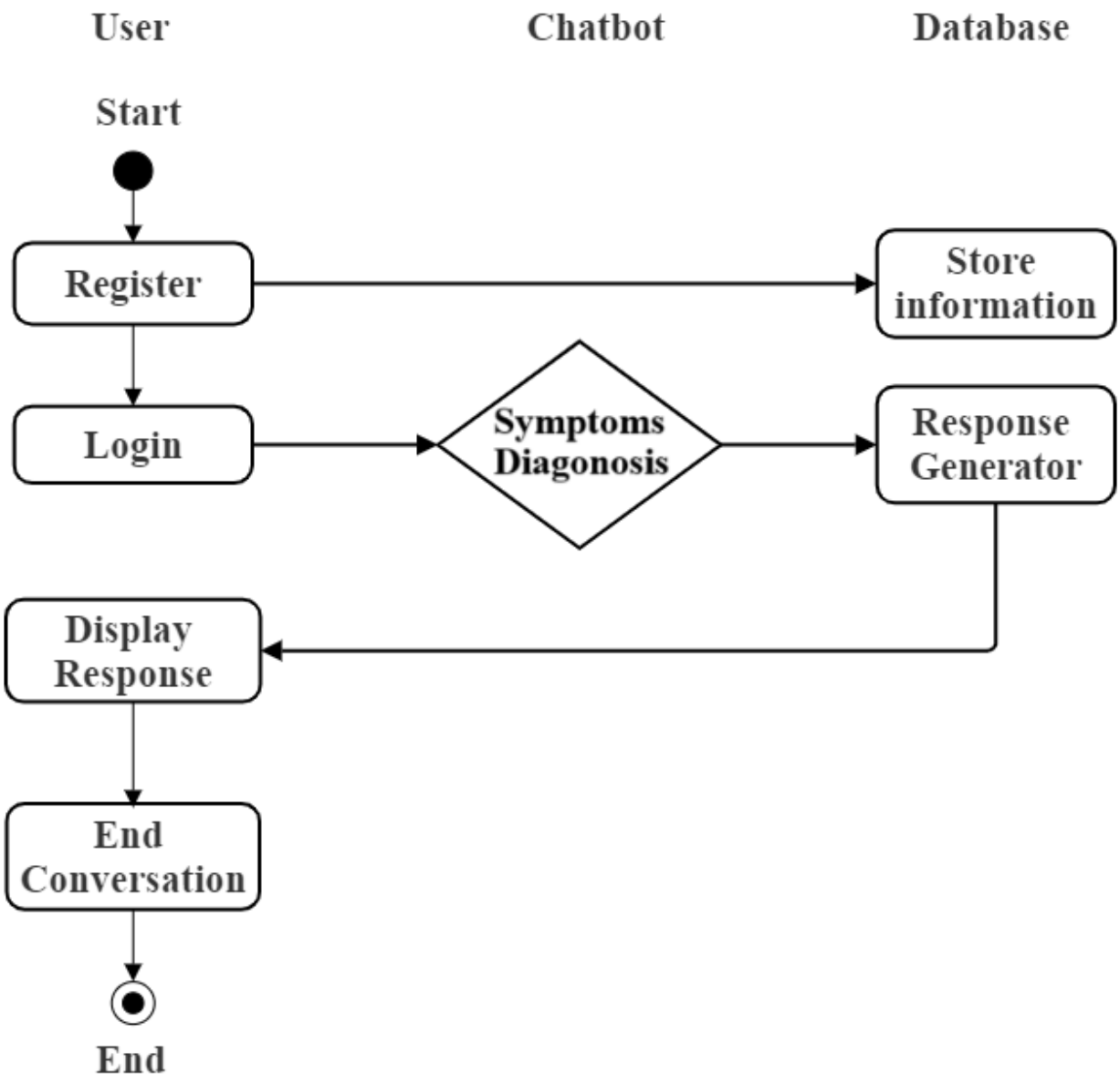

Fig 4.2.4 Activity Diagram for Medimate Messenger

5. IMPLEMENTATION

5.1 CODE FOR THE PROJECT

NEW_LOGIN.py

```
from tkinter import *import os
import tkinter
# Designing window for registration
def destroyPackWidget(parent):
      for e in parent.pack_slaves():
e.destroy()
def register():
   global root,register_screen
destroyPackWidget(root)
register_screen=root
#    register_screen = Toplevel(main_screen)
register_screen.title("Register")
register_screen.geometry("300x250")

   global username
   global password
   global username_entry
   global password_entry
   username = StringVar()
   password = StringVar()

Label(register_screen, text="Please enter the details below", bg="blue").pack()
Label(register_screen, text="").pack()
username_lable = Label(register_screen, text="Username * ")
username_lable.pack()
username_entry = Entry(register_screen, textvariable=username)
username_entry.pack()
password_lable = Label(register_screen, text="Password * ")
password_lable.pack()
password_entry = Entry(register_screen, textvariable=password, show='*')
password_entry.pack()
Label(register_screen, text="").pack()
Button(register_screen, text="Register", width=10, height=1, bg="blue",
command=register_user).pack()
```

```
# Designing window for login
password_not_recog_screen.title("Success")
password_not_recog_screen.geometry("150x100")
Label(password_not_recog_screen, text="Invalid Password ").pack()
Button(password_not_recog_screen, text="OK",
command=delete_password_not_recognised).pack()

# Designing popup for user not found

def user_not_found():
global user_not_found_screen
user_not_found_screen = Toplevel(login_screen)
user_not_found_screen.title("Success")
user_not_found_screen.geometry("150x100")
Label(user_not_found_screen, text="User Not Found").pack()
Button(user_not_found_screen, text="OK",
command=delete_user_not_found_screen).pack()

# Deleting popups

def delete_login_success():
login_success_screen.destroy()
def delete_password_not_recognised():
password_not_recog_screen.destroy()
def delete_user_not_found_screen():
user_not_found_screen.destroy()

# Designing Main(first) window

def main_account_screen(frmmain):
main_screen=frmmain
main_screen.geometry("300x250")
main_screen.title("Account Login")
Label(main_screen,text="Select Your Choice", bg="blue", width="300", height="2",
font=("Calibri", 13)).pack()
   Label(main_screen,text="").pack()
Button(main_screen,text="Login", height="2", width="30", command=login).pack()
   Label(main_screen,text="").pack()
```

```
Button(main_screen,text="Register", height="2", width="30", command=register).pack()
root = tkinter.Tk()
main_account_screen(root)
root.mainloop()
```

QUESTION_DIAGONOIS_TKINTER.py

```
# Importing the libraries
from tkinter import *
from tkinter import messagebox
import os
import webbrowser
import numpy as np
import pandas as pd
class HyperlinkManager:
    def __init__(self, text):
self.text = text
self.text.tag_config("hyper", foreground="blue", underline=1)
self.text.tag_bind("hyper", "<Enter>", self._enter)
self.text.tag_bind("hyper", "<Leave>", self._leave)
self.text.tag_bind("hyper", "<Button-1>", self._click)
self.reset()
    def reset(self):
self.links = {}
    def add(self, action):
        # add an action to the manager.  returns tags to use in
        # associated text widget
        tag = "hyper-%d" % len(self.links)
self.links[tag] = action
        return "hyper", tag
    def _enter(self, event):
self.text.config(cursor="hand2")
    def _leave(self, event):
self.text.config(cursor="")
    def _click(self, event):
        for tag in self.text.tag_names(CURRENT):
            if tag[:6] == "hyper-":
self.links[tag]()
```

```
        return
# Importing the dataset
training_dataset = pd.read_csv('C:\\Users\\Sivadharshini\\Desktop\\Training.csv')
test_dataset = pd.read_csv('C:\\Users\\Sivadharshini\\Downloads\\Testing.csv')

# Slicing and Dicing the dataset to separate features from predictions
X = training_dataset.iloc[:, 0:132].values
Y = training_dataset.iloc[:, -1].values

# Dimensionality Reduction for removing redundancies
dimensionality_reduction = training_dataset.groupby(training_dataset['prognosis']).max()

# Encoding String values to integer constants
from sklearn.preprocessing import LabelEncoder
labelencoder = LabelEncoder()
y = labelencoder.fit_transform(Y)

# Splitting the dataset into training set and test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)

# Implementing the Decision Tree Classifier
from sklearn.tree import DecisionTreeClassifier
classifier = DecisionTreeClassifier()
classifier.fit(X_train, y_train)

# Saving the information of columns
cols     = training_dataset.columns
cols     = cols[:-1]

# Checking the Important features
importances = classifier.feature_importances_
indices = np.argsort(importances)[::-1]
features = cols

# Implementing the Visual Tree
from sklearn.tree import _tree
```

```
# Method to simulate the working of a Chatbot by extracting and formulating questions
def print_disease(node):
self.password_entry = Entry(self, textvariable=self.password, show='*')
self.password_entry.pack()
self.btnRegister=Button(self, text="Register", width=10, height=1, bg="blue",
command=self.register_user)
self.btnRegister.pack()
    def register_user(self):
        file = open(self.username_entry.get(), "w")
file.write(self.username_entry.get() + "\n")
file.write(self.password_entry.get())
file.close()
self.destroyPackWidget(SignUp.main_Root)
self.lblSucess=Label(root, text="Registration Success", fg="green", font=("calibri", 11))
self.lblSucess.pack()
self.btnSucess=Button(root, text="Click Here to proceed",
command=self.btnSucess_Click)
self.btnSucess.pack()
    def btnSucess_Click(self):
self.destroyPackWidget(SignUp.main_Root)
frmQuestion = QuestionDigonosis(SignUp.main_Root)
frmQuestion.pack()
root = Tk()
frmMainForm=MainForm(root)
frmMainForm.pack()
root.mainloop()
```

HEALTHCARE_CHATBODCONSOLE.py

```
# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importing the dataset
training_dataset = pd.read_csv('Training.csv')
test_dataset = pd.read_csv('C:\\Users\\Sivadharshini\\Downloads\\Testing.csv')
```

```
# Slicing and Dicing the dataset to separate features from predictions
X = training_dataset.iloc[:, 0:132].values
#print(X)
  #print(tree_)
feature_name = [
feature_names[i] if i != _tree.TREE_UNDEFINED else "undefined!"
          for i in tree_.feature
       ]
       #print("def tree({}):".format(", ".join(feature_names)))
symptoms_present = []
       def recurse(node, depth):
          indent = "  " * depth
          if tree_.feature[node] != _tree.TREE_UNDEFINED:
             name = feature_name[node]
             threshold = tree_.threshold[node]
print(name + " ?")
ans = input()
ans = ans.lower()
             if ans == 'yes':
val = 1
             else:
val = 0
if  val<= threshold:
                recurse(tree_.children_left[node], depth + 1)
             else:
symptoms_present.append(name)
                recurse(tree_.children_right[node], depth + 1)
          else:
present_disease = print_disease(tree_.value[node])
print( "You may have " +  present_disease )
print()
red_cols = dimensionality_reduction.columns
symptoms_given =
red_cols[dimensionality_reduction.loc[present_disease].values[0].nonzero()]
print("symptoms present  " + str(list(symptoms_present)))
print()
print("symptoms given "  +  str(list(symptoms_given)) )
print()
```

```
confidence_level = (1.0*len(symptoms_present))/len(symptoms_given)
print("confidence level is " + str(confidence_level))
print()
print('The model suggests:')
print()
row = doctors[doctors['disease'] == present_disease[0]]
print('Consult ', str(row['name'].values))
print()
print('Visit ', str(row['link'].values))
            #print(present_disease[0])
recurse(0, 1)
tree_to_code(classifier,cols)
# This section of code to be run after scraping the data
doctors_dataset = pd.read_csv('C:\\Users\\Sivadharshini\\Downloads\\doctors_dataset
(2).csv', names = ['Name', 'Description'])
diseases = dimensionality_reduction.index
diseases = pd.DataFrame(diseases)
doctors = pd.DataFrame()
doctors['name'] = np.nan
doctors['link'] = np.nan
doctors['disease'] = np.nan
doctors['disease'] = diseases['prognosis']
doctors['name'] = doctors_dataset['Name']
doctors['link'] = doctors_dataset['Description']
record = doctors[doctors['disease'] == 'AIDS']
record['name']
record['link']
# Execute the bot and see it in Action
execute_bot()
```

6. TESTING

Testing objectives

- Testing is a process of executing software with the intent of finding errors.
- Good testing has a high probability of finding as-yet-undiscovered errors.
- Successful testing discovers unknown errors.
- If did not find any errors, need to ask whether our testing approach is good.

Testing

The process of evaluating a particular product to determine whether the product contain any defects.

6.1 SOFTWARE TESTING

Software Testing is a process of evaluating a system by manual or automatic means and verify that it satisfies specified requirements or identify differences between expected and actual results.

6.2 BASIC TYPES OF TESTING

- Black Box Testing
- White Box Testing
- Unit Testing
- Integration Testing
- System Testing
- Regression Testing
- Verification and Validation Testing

6.3 TESTING TECHNIQUES USED UNIT TESTING

- Roughly equivalent to chip-level testing for hardware.
- Its testing done to each module, in isolation, to verify its behaviour.

- Typically, the unit test will establish some sort of artificial environment and then invoke routines in the module being tested.
- It then checks the results returned against either some known value or against the results from previous runs of the same test (regression testing).
- When the modules are assembled we can use the same tests to test the system as a whole.

Advantages

- Easy to locate the bug
- Exhaustive testing up to some extent
- Interaction of multiple errors can be avoided.

Reason for Choosing Unit Testing:

- For each function, the testing is done.
- The processes like Query Evaluation, unanimity estimation can be tested.
- The minute changes that may lead to fault can be identified and fixed.

INTEGRATION TESTING

- To check the application against its design.
- To check that the application works.
- Integration Testing
    - Testing of subsystems as a whole
    - Units assumed to have been tested and passed.

Approach

Check program, inter program communications, batch-flow, online dialog.

Typical tests

- Application integration
- Job Streams

- Internal component-to-component interfaces
- Resource availability and contention
- Networks, Internet, communications, etc.
- Support functions
- Interfaces with other application
- Batch Processes
- System-level design features

BLACKBOX TESTING

Advantages

- Black-box testing is a method of software testing that examines the functionality of an application based on the specifications.
- It is also known as Specifications based testing. Independent Testing.
- Team usually performs this type of testing during the software testing life cycle.

Techniques

- Equivalence Class
- Boundary Value Analysis
- Domain Tests
- Orthogonal Arrays
- Decision Tables
- State Models
- tester and developer Exploratory Testing
- All-pairs testing

Advantages

- Efficient when used on large systems.
- Since the are independent of each other, testing is balanced and unprejudiced.
- Tester can be non-technical.

- There is no need for the tester to have detailed functional knowledge of system.
- Tests will be done from an end user's point of view, because the end user should accept the system. (This testing technique is sometimes also called Acceptance testing.)

7. SCREENSHOTS

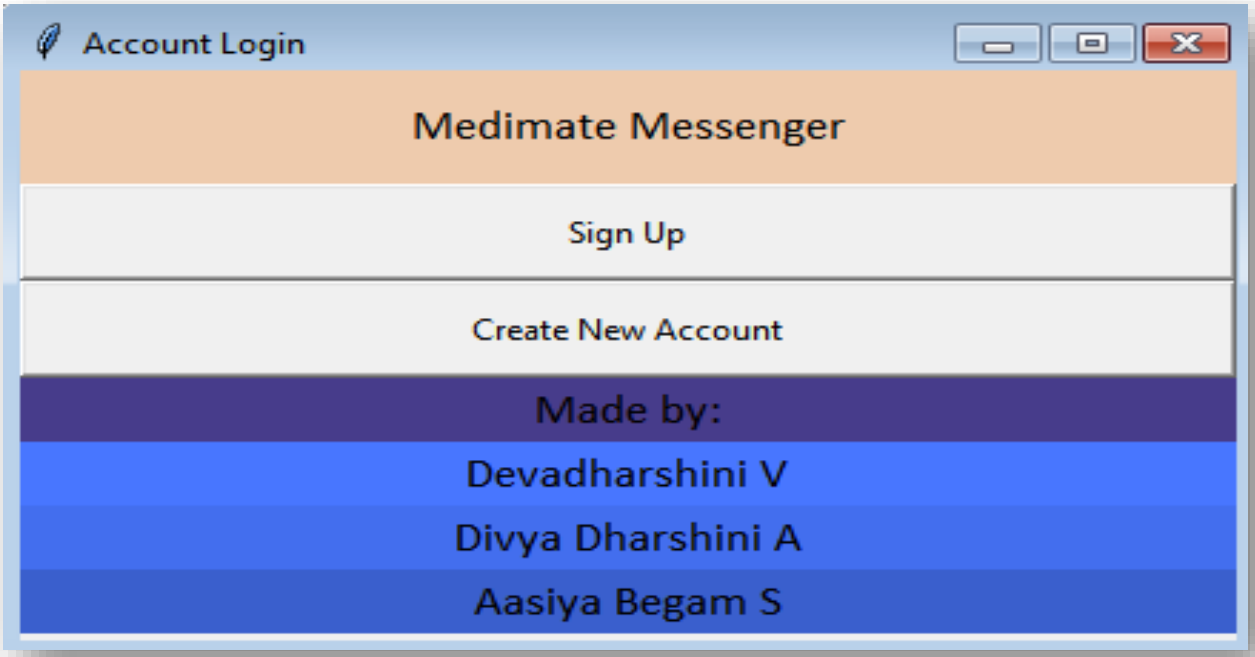

Fig7.1 Screenshot For Home Page

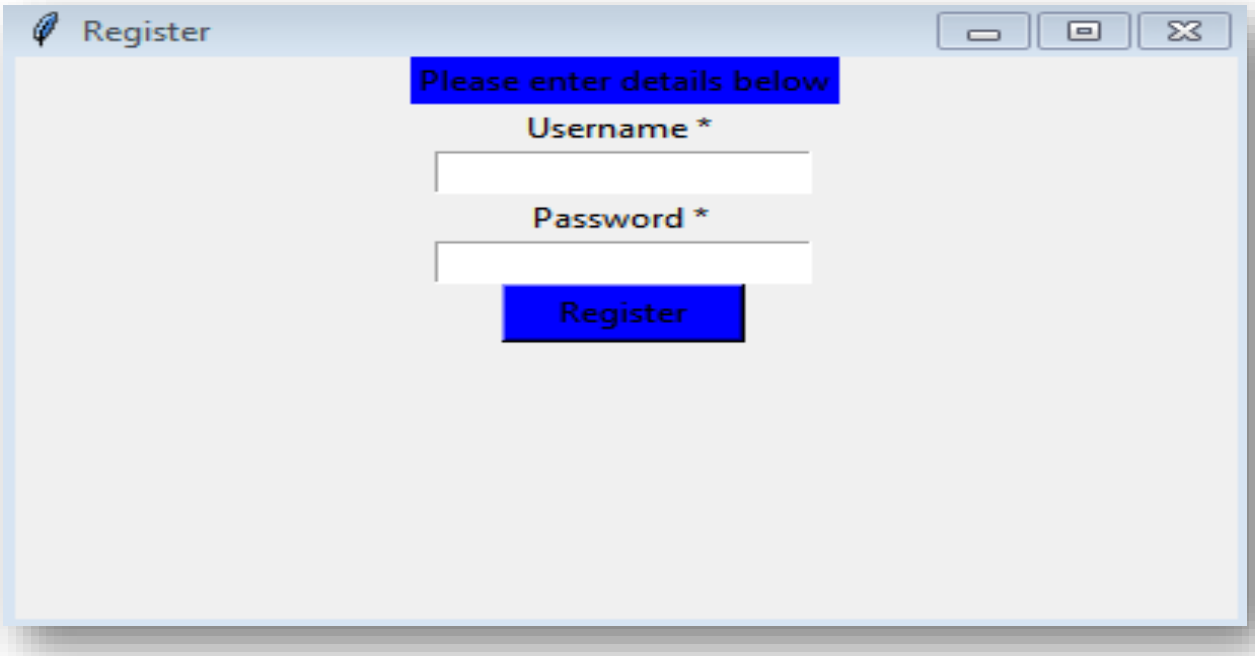

Fig 7.2 Screenshot For Registration Page

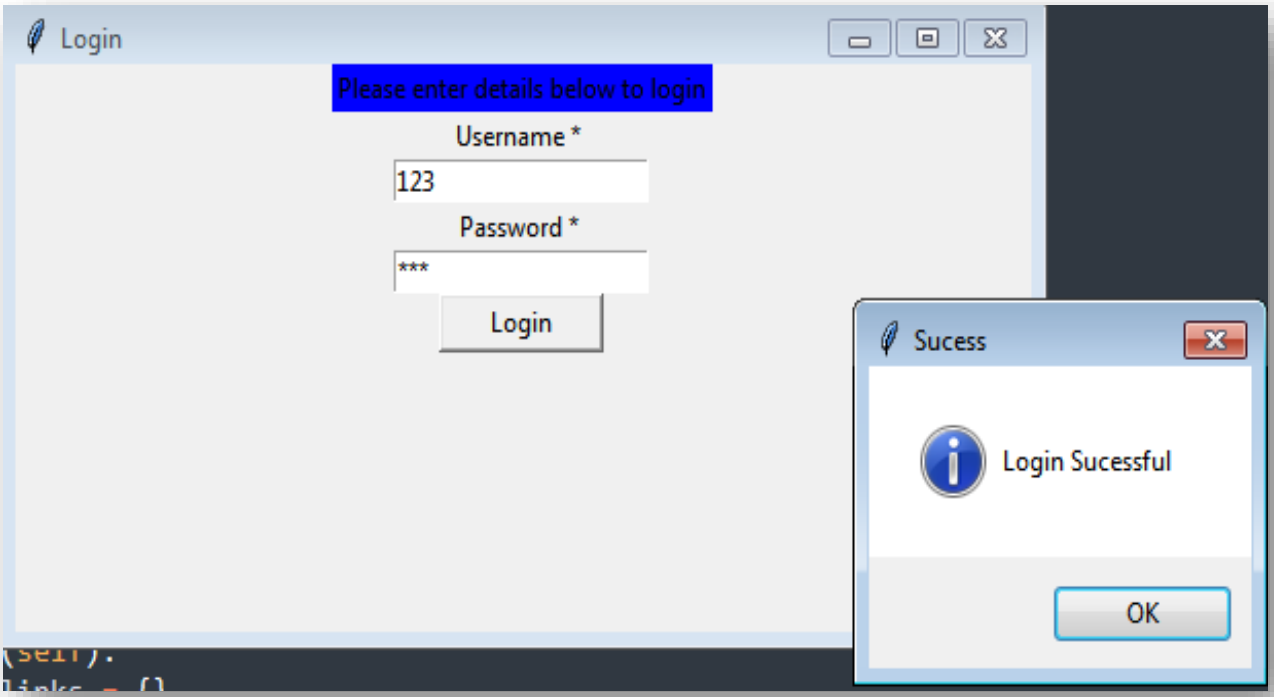

Fig 7.3 Screenshot For Login Page

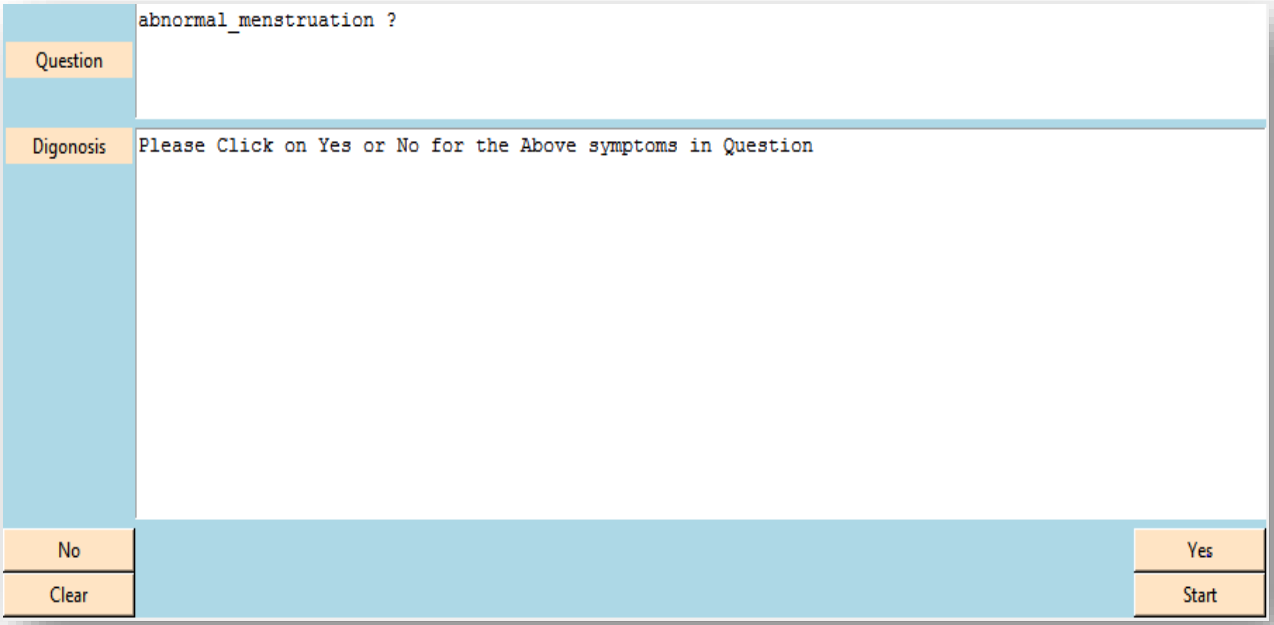

Fig 7.4 Screenshot For Symptoms Diagonosis Page

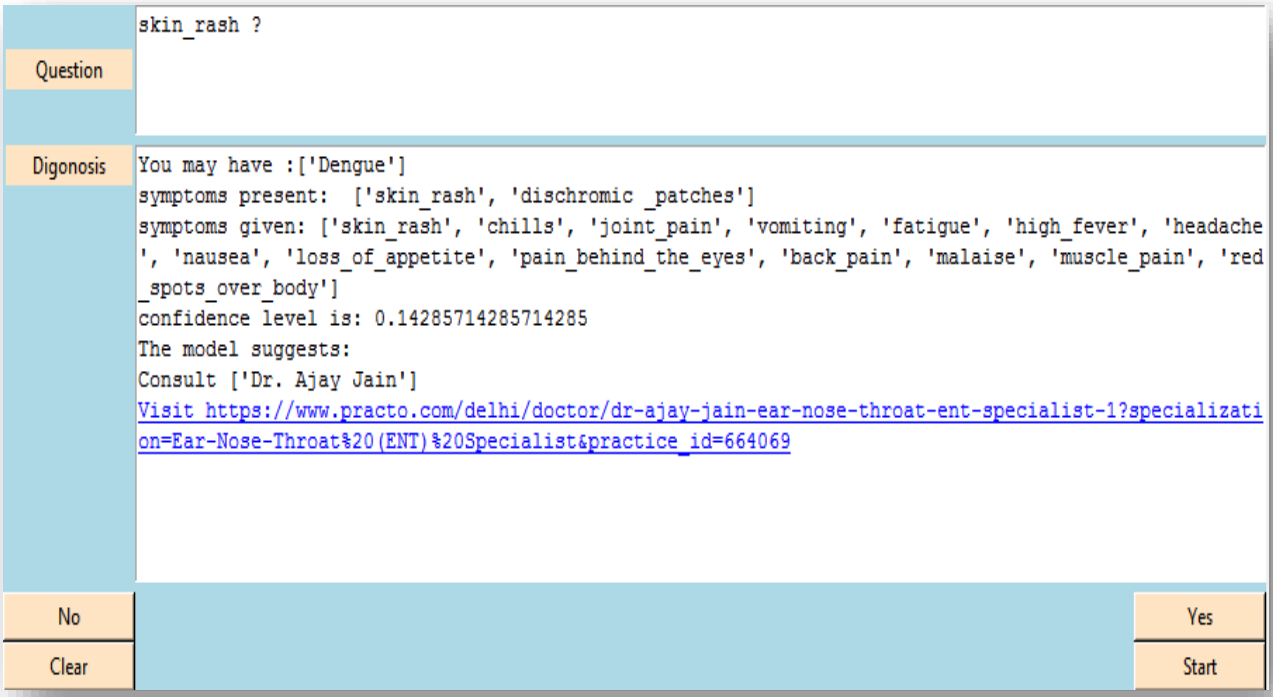

Fig 7.5 Screenshot For Doctor Recommandation Page

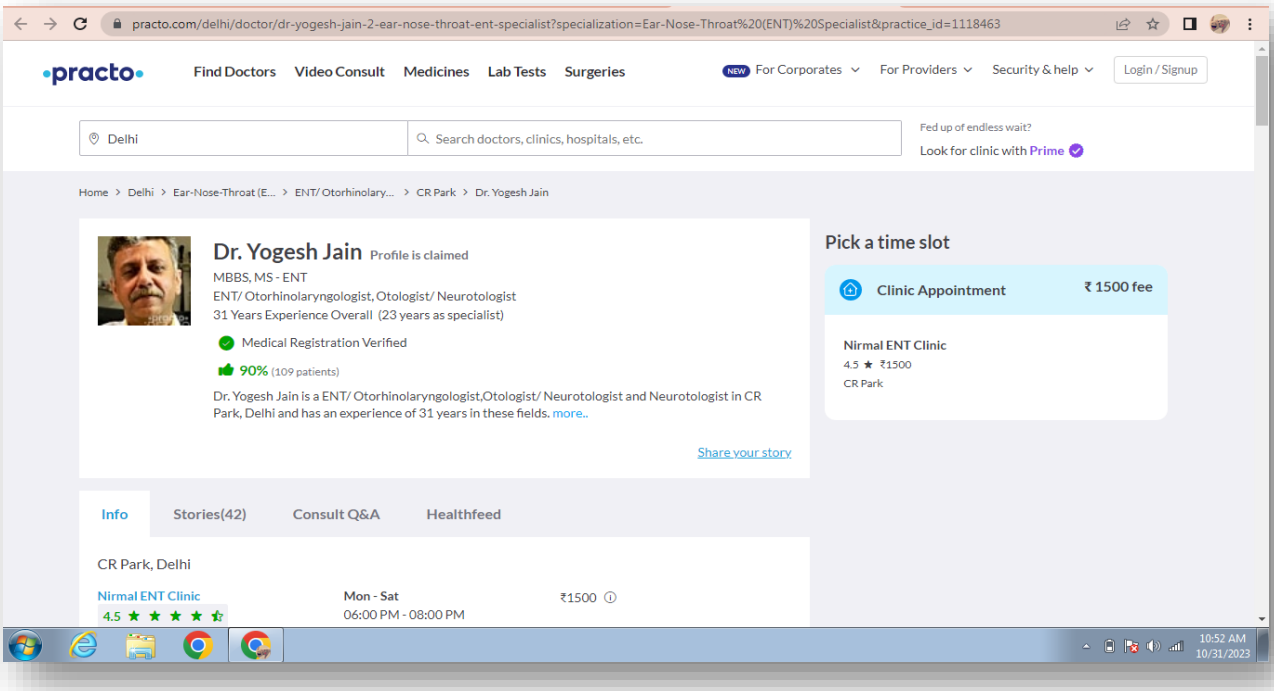

Fig 7.6 Screenshot For Appointment Booking Page

8. CONCLUSION

8.1 CONCLUSION

The development of a Medical Chatbot that combines symptom recognition, doctor recommendations, and AI-driven treatment guidance signifies a promising evolution in the healthcare industry. This technology not only facilitates better healthcare accessibility but also empowers individuals to make informed decisions about their health. As healthcare continues to evolve, such solutions serve as a testament to the potential of AI in improving patient outcomes and making healthcare more patient-centric and efficient. Result of this medical chatbot has the potential to alleviate the burden on healthcare systems by reducing unnecessary emergency room visits and facilitating early intervention for serious medical conditions.

8.2 FUTURE ENHANCEMENT

- The model is only accessible via webapp, in future we will be extending this app for mobile application and also introducing some other features and model alteration by the user.
- Allow users to call emergency services or send out alrets in case of critical medical situations

9. APPENDIX

9.1 ABOUT THE SOFTWARE

Python is an interpreted, high-level and general-purpose programming language. Created by Guido van Rossum and first released in 1991, Python's design philosophy emphasizes code readability with its notable use of significant whitespace. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects. **Python** is used for web development, AI, machine learning, operating systems, mobile application development, and video games. Also it has high security measures. why we are using this for our project. It has high confidentiality and can very hard to crack the data within the system.

The language's core philosophy is including aphorisms such as:

- Beautiful is better than ugly
- Explicit is better than implicit
- Simple is better than complex
- Complex is better than complicated
- Readability counts

Python is meant to be an easily readable language. Its formatting is visually uncluttered, and it often uses English keywords where other languages use punctuation. Unlike many other languages, it does not use curly brackets to delimit blocks, and semicolons after statements are optional. It has fewer syntactic exceptions and special cases than C or Pascal. Python uses whitespace indentation, rather than curly brackets or keywords, to delimit blocks.

An increase in indentation comes after certain statements; a decrease in indentation signifies the end of the current block. Thus, the program's visual structure accurately represents the program's semantic structure. This feature is also sometimes termed the off-side rule.

Statements and control flow

Python's statements include (among others):

The assignment statement (token '=', the equals sign). This operates differently than in traditional imperative programming languages, and this fundamental mechanism (including the nature of Python's version of variables) illuminates many other features of the language.

Assignment in C, e.g., x = 2, translates to "typed variable name x receives a copy of numeric value 2". The (right-hand) value is copied into an allocated storage location for which the (left-hand) variable name is the symbolic address.

The memory allocated to the variable is large enough (potentially quitelarge) for the declared type. In the simplest case of Python assignment, using the same example, x = 2, translates to "(generic) name x receives a reference to a separate, dynamically allocated object of numeric (int) type of value 2." This is termed binding the name to the object. Since the name's storage location doesn't contain the indicated value, it is improper to call it a variable. Names may be subsequently rebound at any time to objects of greatly varying types, including strings, procedures, complex objects with data and methods, etc.

Successive assignments of a common value to Assignment in C, e.g., x = 2, translates to "typed variable name x receivesa copy of numeric value 2". The (right-hand) value is copied into anallocated storage location for which the (left-hand) variable name is thesymbolic address. The memory allocated to the variable is (potentially quitelarge) for the declared type. In the simplest case of Python assignment, the same example, x = 2, translates to "(generic) name x receives areference to a separate, dynamically allocated object of numeric (INT) type of value 2." This is termed binding the name to the object.

Since the name'sstorage location doesn't contain the indicated value. it is improper to call ita variable. Names may be subsequently rebound at any time to objects ofgreatly varying types, including strings, procedures, complex objects with data and methods, etc. Successive assignments of a common Assignment in C, e.g., x = 2, translates to "typed variable name x receives a copy of numeric value 2". The (right-hand) value is

copied into an allocated storage location for which the (left-hand) variable name is the symbolic address. The memory allocated to the variable is large (potentially quitelarge) for the declared type. In the simplest case of Python assignment, using the same example, x = 2, translates to "(generic) name x receives a reference to a separate, dynamically allocated object of numeric (INT) type of value 2." This is termed binding the name to the object. Since the name's storage location doesn't contain the indicated value, it is improper to call it a variable.

The assert statement, used during debugging to check for conditions that ought to apply. The yield statement, which returns a value from a generator function. FromPython 2.5, yield is also an operator. This form is used to implement co routines. The import statement, which is used to import modules whose functions orvariables can be used in the current program. There are three ways of using: import: import <module name> [as <alias>] or from <module name>import * or from <module name> import <definition 1> [as <alias 1>], <definition 2> [as <alias 2>], ... The print statement was changed to the print () function in Python 3. Pythondoes not support tail call optimization or first-class continuations, and, according out of the generator. From Python 2.5, it is possible to pass information back into a generator function, and from Python 3.3, the information can be passed through multiple stack levels.to Guido van Rossum, it never will. However, better support for co routine-like functionality is provided in 2.5, by extending Python's generators. Before 2.5, generators were lazy iterators; information passed unidirectional.

10.
REFERENCES

[1] S. Chakraborty and L. Dey, " The implementation of AI and AI-empowered imaging system to fight against COVID-19—A review, " in Smart Health-care System Design: Security and Privacy Aspects. 2022, Dec.

[2] E. Tebenkov and I. Prokhorov, "Machine learning algorithms for teaching AI chat bots," Proc. Comput. Sci., vol. 190, pp. 735–744, Jan. 2021.

[3] M. M. Hossain, S. Krishna Pillai, S. E. Dansy, and A. A. Bilong, "Mr. Dr. Health-assistant chatbot," Int. J. Artif. Intell., vol. 8, no. 2, pp. 58–73, Dec. 2021.

[4] K. H. Koundinya, A. K. Palakurthi, V. Putnala, and K. A. Kumar, "Smart college chatbot using ML and Python," in Proc. Int. Conf. Syst., Comput., Automat. Netw. (ICSCAN), Jul. 2020, pp. 1–5.

[5] A. Kumar, P. K. Meena, D. Panda, and M. Sangeetha, "Chatbot in Python,"Int. Res. J. Eng. Technol., vol. 6, no. 11, 2019.

[6] S. Raj and K. Raj, Building Chatbots With Python. New York, NY, USA:Apress, 2019.

[7] S. A. Sheikh, "Artificial intelligence based chatbot for human resource using deep learning," Ph.D. dissertation, Dept. Comput. Sci. Eng., Manipal Univ., Manipal, India, 2019.

[8] A. S. Lokman and M. A. Ameedeen, "Modern chatbot systems: A technical review," in Proc. Future Technol. Conf. Cham, Switzerland: Springer,Nov. 2018, pp. 1012–1023.

[9] R. Dharwadkar and N. A. Deshpande, "A medical chatbot," Int. J. Comput. Trends Technol., vol. 60, no. 1, pp. 41–45, 2018.

[10] D. Madhu, C. J. N. Jain, E. Sebastain, S. Shaji, and A. Ajayakumar, "A novel approach for medical assistance using trained chatbot," in Proc. Int. Conf. Inventive Commun. Comput. Technol. (ICICCT), Mar. 2017.